

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

SKILL – WEEK 10

Name: S Svava Haasini

Roll No: 2520090170

Task 1: Signal Registration and Event Handling

Question:

Register handlers, configure signal actions, handle interrupts, manage signal context, verify stability, and test signal events.

Source Code:

```
GNU nano 7.2 T1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

volatile sig_atomic_t sigint_count = 0;
volatile sig_atomic_t sigusr1_count = 0;

void handler(int sig)
{
    if (sig == SIGINT)
    {
        sigint_count++;
        printf("\nSIGINT received. Count: %d\n", sigint_count);
    }
    else if (sig == SIGUSR1)
    {
        sigusr1_count++;
        printf("SIGUSR1 received. Count: %d\n", sigusr1_count);
    }
}

int main(void)
{
    struct sigaction sa;

    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1)
    {
        perror("sigaction SIGINT");
        return 1;
    }

    if (sigaction(SIGUSR1, &sa, NULL) == -1)
    {
        perror("sigaction SIGUSR1");
        return 1;
    }

    printf("Process PID: %d\n", getpid());
    printf("Signal handlers registered.\n");

    printf("\nSending SIGUSR1 to this process...\n");
    raise(SIGUSR1);

    printf("\nNow testing SIGINT automatically...\n");
    raise(SIGINT);

    printf("\nSignal event test completed.\n");
    printf("SIGUSR1 count: %d\n", sigusr1_count);
    printf("SIGINT count : %d\n", sigint_count);

    return 0;
}
```

Output:

```

pwd
/home/haasini/OSSP_upload/Skill_Week10
haasini@SSH:~/OSSP_upload/Skill_Week10$ nano T1.c
haasini@SSH:~/OSSP_upload/Skill_Week10$ gcc -Wall -Wextra T1.c -o T1
haasini@SSH:~/OSSP_upload/Skill_Week10$ ./T1
Process PID: 1175
Signal handlers registered.

Sending SIGUSR1 to this process...
SIGUSR1 received. Count: 1

Now testing SIGINT automatically...

SIGINT received. Count: 1

Signal event test completed.
SIGUSR1 count: 1
SIGINT count : 1
haasini@SSH:~/OSSP_upload/Skill_Week10$ |

```

Observations:

1. volatile sig_atomic_t counters track signal delivery safely, maintaining separate counts for SIGINT and SIGUSR1 events.
2. The handler() function checks signal number and increments the appropriate counter when signals arrive, demonstrating signal-specific handling.
3. sigaction() registers the handler with signal masking and flag configuration, replacing the default handler with custom behavior.
4. sigemptyset(&sa.sa_mask) initializes the signal mask associated with the sigaction handler configuration, controlling signal blocking during handler execution.
5. The program displays 'Process PID' and 'Signal handlers registered' to confirm handler installation.
6. raise(SIGUSR1) and raise(SIGINT) send signals to the process, triggering the registered handler immediately.
7. The output shows both 'SIGUSR1 received. Count: 1' and 'SIGINT received. Count: 1', confirming each handler executed.
8. The counters are incremented safely using atomic operations, maintaining accurate delivery counts across multiple signal events.

Task 2: Interrupt Handling and Signal Forwarding

Question:

Capture SIGINT, forward signals, protect shell process, terminate foreground jobs, update job states, and test interrupt handling.

Source Code:

```
GNU nano 7.2 T2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

volatile sig_atomic_t foreground_pid = -1;

void sigint_handler(int sig)
{
    (void)sig;

    if (foreground_pid > 0)
    {
        printf("\nShell captured SIGINT.\n");
        printf("Forwarding SIGINT to foreground job %d...\n",
            foreground_pid);

        kill(foreground_pid, SIGINT);
    }
}

int main(void)
{
    struct sigaction sa;

    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1)
    {
        perror("sigaction");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        signal(SIGINT, SIG_DFL);

        printf("Foreground job started. PID: %d\n", getpid());
        fflush(stdout);

        while (1)
        {
            // ... (foreground job logic) ...
        }
    }
}
```

```

        while (1)
        {
            printf("Foreground job running...\n");
            fflush(stdout);
            sleep(1);
        }
    }

    foreground_pid = pid;

    printf("Shell PID: %d\n", getpid());
    printf("Foreground job PID: %d\n", pid);

    sleep(2);

    printf("\nSimulating Ctrl+C using SIGINT...\n");

    raise(SIGINT);

    int status;
    waitpid(pid, &status, 0);

    if (WIFSIGNALED(status))
    {
        printf("Foreground job terminated by signal %d.\n",
               WTERMSIG(status));
    }

    foreground_pid = -1;

    printf("Shell process is still running.\n");
    printf("Interrupt handling test completed.\n");

    return 0;
}

```

Output:

```

/home/haasini/OSSP_upload/Skill_Week10
haasini@SSH:~/OSSP_upload/Skill_Week10$ nano T1.c
haasini@SSH:~/OSSP_upload/Skill_Week10$ gcc -Wall -Wextra T1.c -o T1
haasini@SSH:~/OSSP_upload/Skill_Week10$ ./T1
Process PID: 1175
Signal handlers registered.

Sending SIGUSR1 to this process...
SIGUSR1 received. Count: 1

Now testing SIGINT automatically...

SIGINT received. Count: 1

Signal event test completed.
SIGUSR1 count: 1
SIGINT count : 1
haasini@SSH:~/OSSP_upload/Skill_Week10$ nano T2.c
haasini@SSH:~/OSSP_upload/Skill_Week10$ gcc -Wall -Wextra T2.c -o T2
haasini@SSH:~/OSSP_upload/Skill_Week10$ ./T2
Shell PID: 1377
Foreground job started. PID: 1378
Foreground job PID: 1378
Foreground job running...
Foreground job running...

Simulating Ctrl+C using SIGINT...

Shell captured SIGINT.
Forwarding SIGINT to foreground job 1378...
Foreground job terminated by signal 2.
Shell process is still running.
Interrupt handling test completed.
haasini@SSH:~/OSSP_upload/Skill_Week10$ |

```

Observations:

1. `volatile sig_atomic_t foreground_pid` stores the PID of the currently active foreground job for signal forwarding.
2. `sigint_handler()` checks if a foreground job is running (`foreground_pid > 0`) and forwards `SIGINT` to that job using `kill()`.
3. The handler protects the shell process itself by not executing default termination behavior when `SIGINT` is caught.
4. `sigaction(SIGINT, &sa, NULL)` in the parent installs the custom `sigint_handler`, intercepting `Ctrl+C` keypresses.
5. The child process uses `signal(SIGINT, SIG_DFL)` to restore default `SIGINT` behavior, allowing it to terminate normally when `SIGINT` is delivered.
6. `fork()` creates a child process that becomes the foreground job, running indefinitely with `sleep()` loops.
7. The parent process stores the child's PID in `foreground_pid` and simulates `Ctrl+C` by calling `raise(SIGINT)`.
8. The shell's `sigint_handler` captures `SIGINT` and forwards it via `kill(foreground_pid, SIGINT)` to terminate the foreground job.
9. The output shows 'Shell captured `SIGINT`' and 'Forwarding `SIGINT` to foreground job', confirming signal interception and forwarding.

Task 3: SIGTSTP/SIGCONT for Job Suspension

Question:

Capture SIGTSTP, suspend processes, update job table, preserve state, resume later, and test suspension workflow.

Source Code:

```
GNU nano 7.2 T3.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

typedef struct
{
    int job_id;
    pid_t pid;
    char *state;
} Job;

int main(void)
{
    Job job;

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child job started. PID: %d\n", getpid());
        fflush(stdout);

        for (int i = 1; i <= 6; i++)
        {
            printf("Child working... step %d\n", i);
            fflush(stdout);
            sleep(1);
        }

        printf("Child completed normally.\n");
        exit(0);
    }

    job.job_id = 1;
    job.pid = pid;
    job.state = "Running";

    sleep(2);

    printf("\nSending SIGTSTP to job %d...\n", job.job_id);

    kill(pid, SIGTSTP);

    int status;

    waitpid(pid, &status, WUNTRACED);

    if (WIFSTOPPED(status))
    {
        job.state = "Stopped";
    }
}
```

```

if (WIFSTOPPED(status))
{
    job.state = "Stopped";

    printf("Job stopped by signal %d.\n",
        WSTOPSIG(status));
}

printf("\nJob Table:\n");
printf("JOB\tPID\tSTATE\n");
printf("%d\t%d\t%s\n",
    job.job_id,
    job.pid,
    job.state);

sleep(2);

printf("\nResuming job with SIGCONT...\n");
kill(pid, SIGCONT);

job.state = "Running";

printf("\nUpdated Job Table:\n");
printf("JOB\tPID\tSTATE\n");
printf("%d\t%d\t%s\n",
    job.job_id,
    job.pid,
    job.state);

waitpid(pid, &status, 0);

job.state = "Completed";

printf("\nFinal Job Table:\n");
printf("JOB\tPID\tSTATE\n");
printf("%d\t%d\t%s\n",
    job.job_id,
    job.pid,
    job.state);

printf("\nSIGTSTP/SIGCONT test completed.\n");

return 0;
}

```

Output:

```

Signal event test completed.
SIGUSR1 count: 1
SIGINT count : 1
haasini@SSH:~/OSSP_upload/Skill_Week10$ nano T2.c
haasini@SSH:~/OSSP_upload/Skill_Week10$ gcc -Wall -Wextra T2.c -o T2
haasini@SSH:~/OSSP_upload/Skill_Week10$ ./T2
Shell PID: 1377
Foreground job started. PID: 1378
Foreground job PID: 1378
Foreground job running...
Foreground job running...

Simulating Ctrl+C using SIGINT...

Shell captured SIGINT.
Forwarding SIGINT to foreground job 1378...
Foreground job terminated by signal 2.
Shell process is still running.
Interrupt handling test completed.
haasini@SSH:~/OSSP_upload/Skill_Week10$ nano T3.c
haasini@SSH:~/OSSP_upload/Skill_Week10$ gcc -Wall -Wextra T3.c -o T3
haasini@SSH:~/OSSP_upload/Skill_Week10$ ./T3
Child job started. PID: 1484
Child working... step 1
Child working... step 2

Sending SIGTSTP to job 1...
Job stopped by signal 20.

Job Table:
JOB      PID      STATE
1        1484     Stopped

Resuming job with SIGCONT...

Updated Job Table:
JOB      PID      STATE
1        1484     Running
Child working... step 3
Child working... step 4
Child working... step 5
Child working... step 6
Child completed normally.

Final Job Table:
JOB      PID      STATE
1        1484     Completed

SIGTSTP/SIGCONT test completed.
haasini@SSH:~/OSSP_upload/Skill_Week10$ |

```

Observations:

1. A Job struct maintains job_id, pid, and state to track each job's current status across suspension and resumption cycles.
2. fork() creates a child process that runs iterative work (6 steps with sleep delays) simulating a long-running background job.
3. The parent stores the child's PID and initial state as 'Running', establishing the initial job table entry.
4. kill(pid, SIGTSTP) sends SIGTSTP to the running child, suspending it without terminating execution (state becomes 'Stopped').
5. waitpid(pid, &status;, WUNTRACED) captures the stopped state change, and WIFSTOPPED(status) detects the suspension.
6. The job table displays state transition from 'Running' to 'Stopped', confirming suspension and state preservation.
7. kill(pid, SIGCONT) sends SIGCONT to resume the stopped job, which resumes execution from the suspension point.
8. The job table after resumption shows state changing back to 'Running', and the child completes remaining work steps.
9. Final job table shows 'Completed' state after the child exits, demonstrating full lifecycle: Running → Stopped → Running → Completed.

Task 4: Process Groups and Terminal Control

Question:

Create process groups, assign group IDs, transfer terminal control, restore ownership, coordinate signals, and validate behavior.

Source Code:

```
GNU nano 7.2 T4.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

int main(void)
{
    if (!isatty(STDIN_FILENO))
    {
        printf("Standard input is not a terminal.\n");
        return 1;
    }

    pid_t shell_pgid = getpgid();

    printf("Parent PID : %d\n", getpid());
    printf("Parent PGID: %d\n", shell_pgid);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        setpgid(0, 0);

        printf("Child PID : %d\n", getpid());
        printf("Child PGID : %d\n", getpgid());
        fflush(stdout);

        sleep(3);

        printf("Foreground process-group job completed.\n");
        fflush(stdout);

        exit(0);
    }

    setpgid(pid, pid);

    signal(SIGTTOU, SIG_IGN);

    printf("\nCreated process group %d.\n", pid);

    if (tcsetpgrp(STDIN_FILENO, pid) == -1)
    {
        perror("tcsetpgrp child");
        kill(pid, SIGTERM);
        waitpid(pid, NULL, 0);
        return 1;
    }

    printf("Terminal control transferred to child group.\n");

    waitpid(pid, NULL, 0);

    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
    {
        perror("tcsetpgrp parent");
        return 1;
    }

    printf("Terminal control restored to parent group.\n");
    printf("Process-group test completed.\n");

    return 0;
}
```

Output:

```

SIGTSTP/SIGCONT test completed.
haasini@SSH:~/OSSP_upload/Skill_Week10$ nano T4.c
haasini@SSH:~/OSSP_upload/Skill_Week10$ gcc -Wall -Wextra T4.c -o T4
haasini@SSH:~/OSSP_upload/Skill_Week10$ ./T4
Parent PID : 1572
Parent PGID: 1572

Created process group 1573.
Terminal control transferred to child group.
Child PID   : 1573
Child PGID  : 1573
Foreground process-group job completed.
Terminal control restored to parent group.
Process-group test completed.
haasini@SSH:~/OSSP_upload/Skill_Week10$ |

```

Observations:

1. `isatty(STDIN_FILENO)` checks if standard input is connected to a terminal, protecting the process from non-terminal environments.
2. `getpgrp()` and `getpgid(0)` retrieve the parent process's process group ID, establishing the initial foreground process group.
3. `fork()` creates a child process that will become the leader of a new process group for terminal control demonstration.
4. `setpgid(0, 0)` in the child creates a new process group with the child as the group leader, isolating it from the parent's group.
5. `tcsetpgrp(STDIN_FILENO, pid)` transfers terminal control from the parent process group to the child process group.
6. `signal(SIGTTOU, SIG_IGN)` prevents the parent from being stopped by `SIGTTOU` while performing terminal-control operations.
7. The parent calls `tcsetpgrp()` with the `shell_pgid` to restore terminal control after the child process completes.
8. The output shows process group transitions: parent PGID matches shell PGID initially, child creates new group (1573), terminal control transfers.
9. Terminal control restoration message confirms `tcsetpgrp()` successfully returned ownership to parent, completing the process-group test.